CS 380: ARTIFICIAL INTELLIGENCE

Lecture 8: Logical Agents

Summary so far:

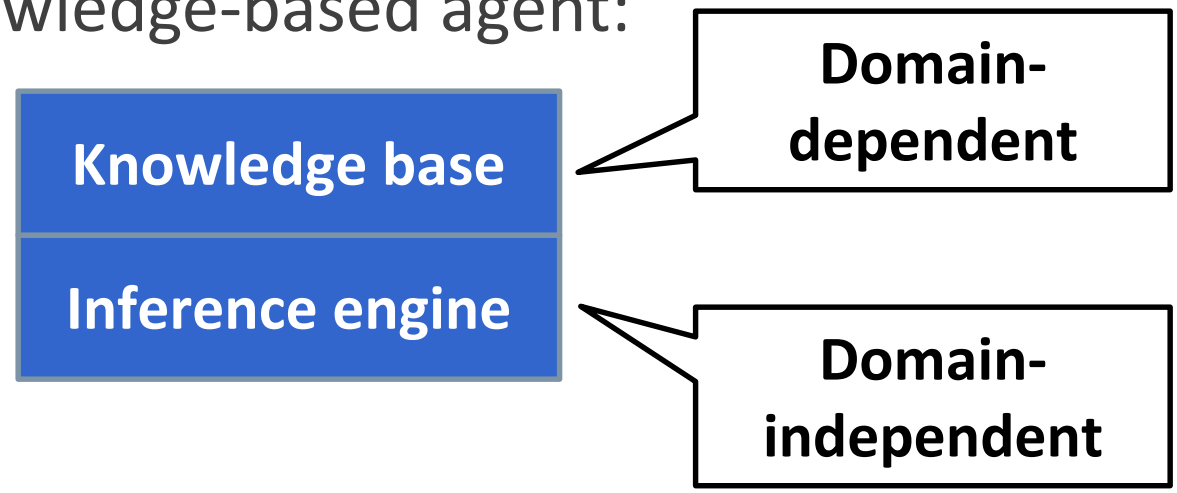
- What is AI?
  - Rational Agents
- Problem solving:
  - Systematic search
    - Uninformed search: DFS, BFS, ID
    - Informed search: heuristics, Greedy Search, A*
  - Local Search: Hill Climbing, Simulated Annealing
  - Adversarial Search: Minimax, alpha-beta, Monte Carlo Tree Search
- **Next: Knowledge Representation + Reasoning**

Knowledge

- Knowledge base:
  - When you think of “knowledge”, what do you think of?

Knowledge

- Knowledge base:
  - For our purposes: A set of **sentences** in a **formal language**
- Knowledge-based agent:



- "Declarative programming"

Declarative Programming

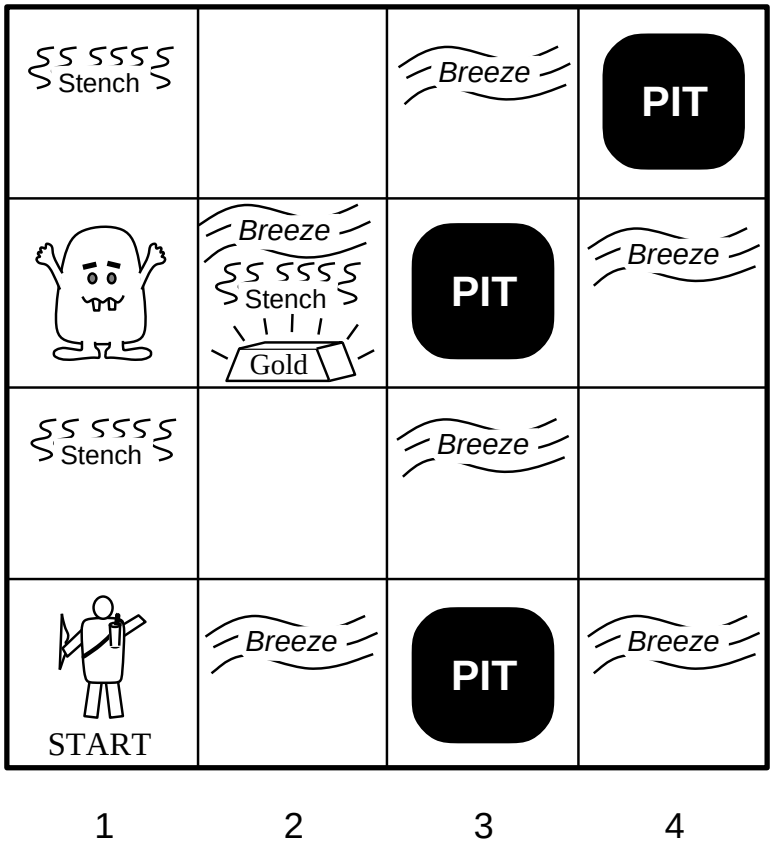
- Tell an agent **WHAT** to do, but not **HOW**.
- Encode all the necessary knowledge, and let the agent *infer* how to achieve the goals.
- For example, instead of programming a specific sorting algorithm, we could tell an agent:
  - Here's a set of numbers.
  - Now I want them sorted, which means that for every adjacent numbers *a* and *b*, they must satisfy *a* <= *b*
  - You (the agent) figure out how ☺
- In the limit, this is really hard

Wumpus World

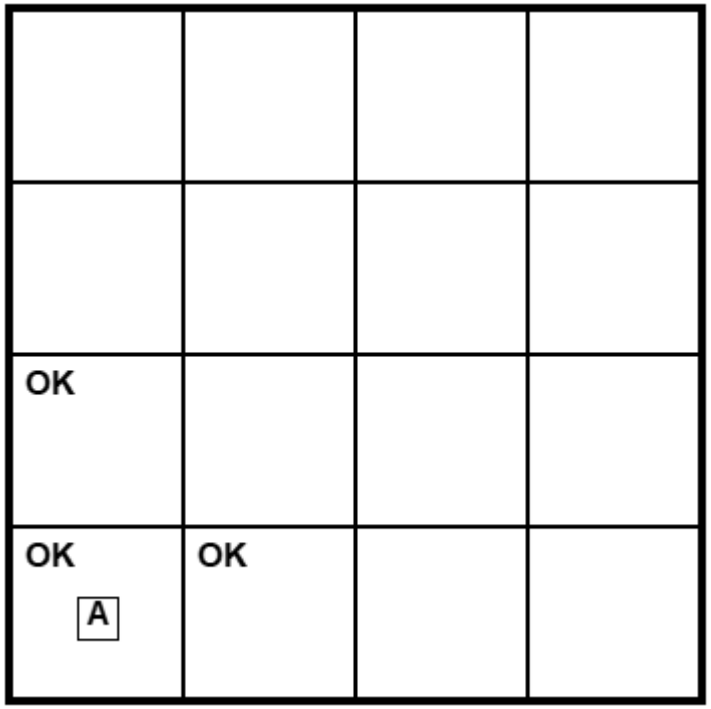
- There's a cave with a terrible "Wumpus" that eats anyone who enters its room ☹
- The Wumpus can be shot with an arrow if you're facing the right direction ☺
- … but you only have 1 arrow ☹
- Some rooms have bottomless pits that trap you ☹
- So why are you poking around Wumpus World? There's gold in one room!! ☺ ☺ ☺

Wumpus World PEAS Description

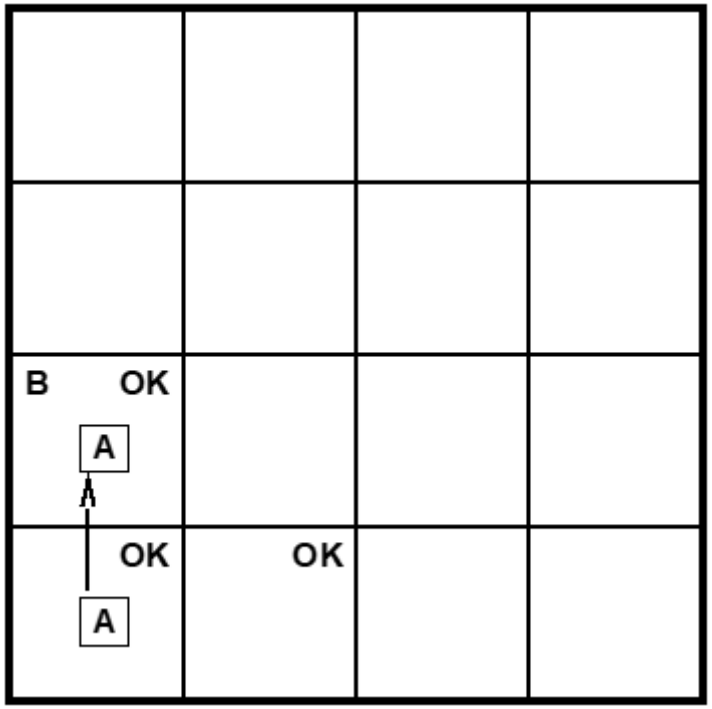
- Performance measure
  - Gold +1000, death -1000,
  - -1 per step, -10 for using arrow
- Environment
  - Squares adjacent to wumpus are smelly
  - Squares adjacent to pit are breezy
  - Glitter iff gold is in the same square
  - Shooting kills wumpus if you are facing it
  - Shooting uses up the only arrow
  - Grabbing picks up gold if in same square
  - Releasing drops gold in same square
- Actuators
  - Left turn, Right turn, Forward, Grab, Release, Shoot
- Sensors - Breeze, Glitter, Smell



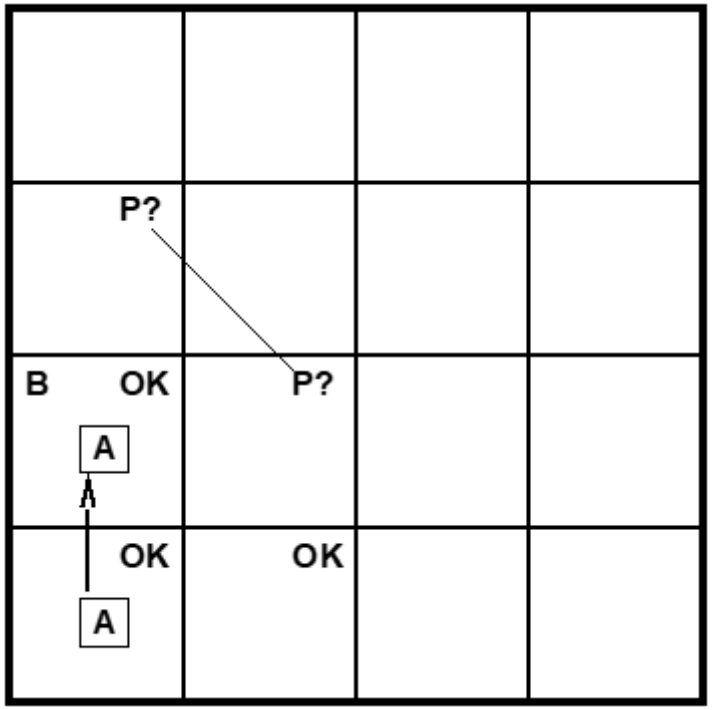
Exploring the Wumpus World



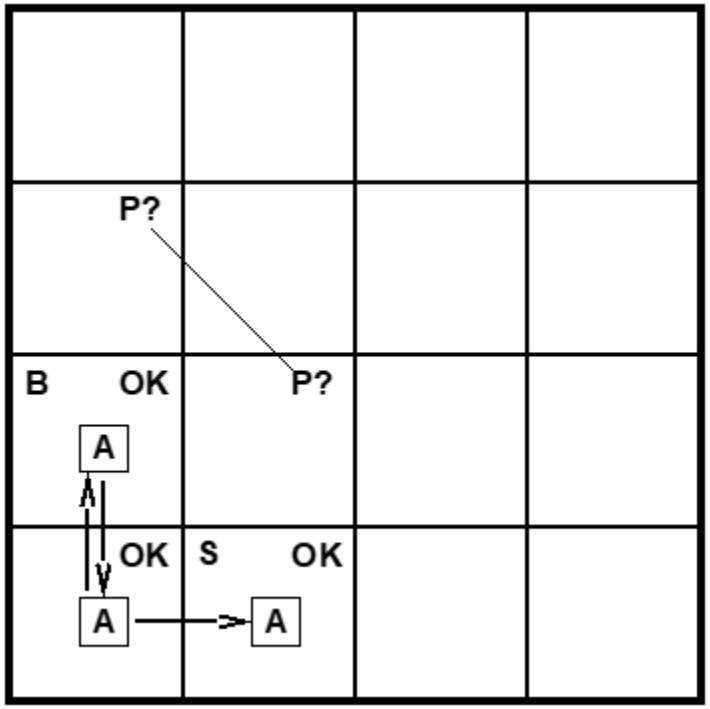
Exploring the Wumpus World



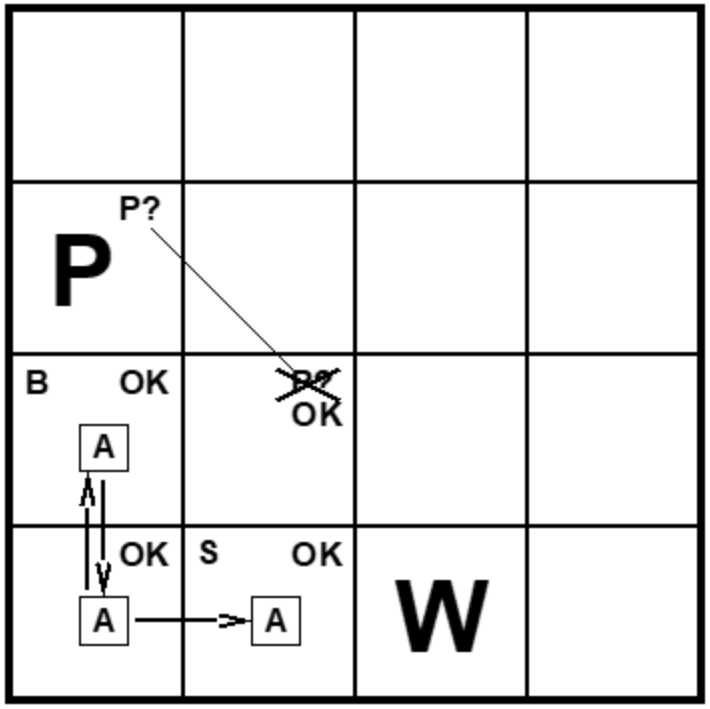
Exploring the Wumpus World



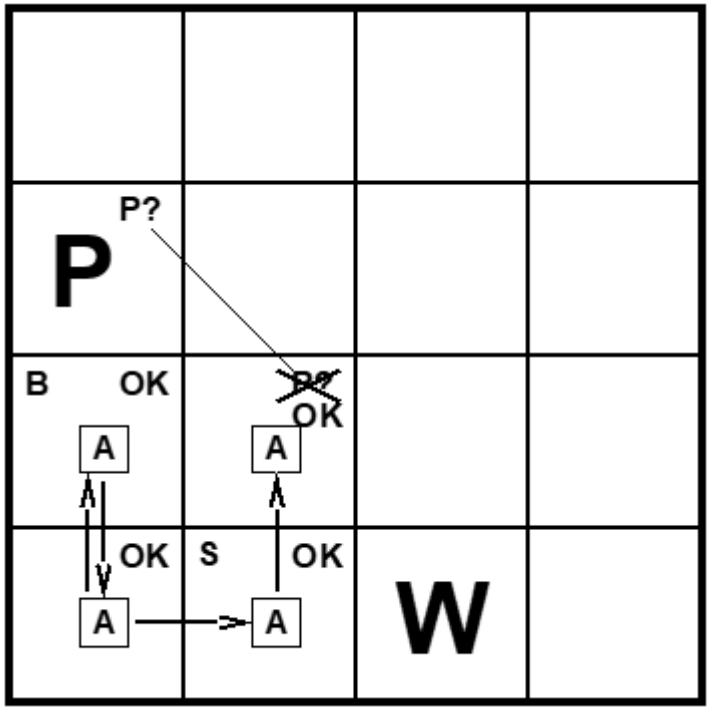
Exploring the Wumpus World



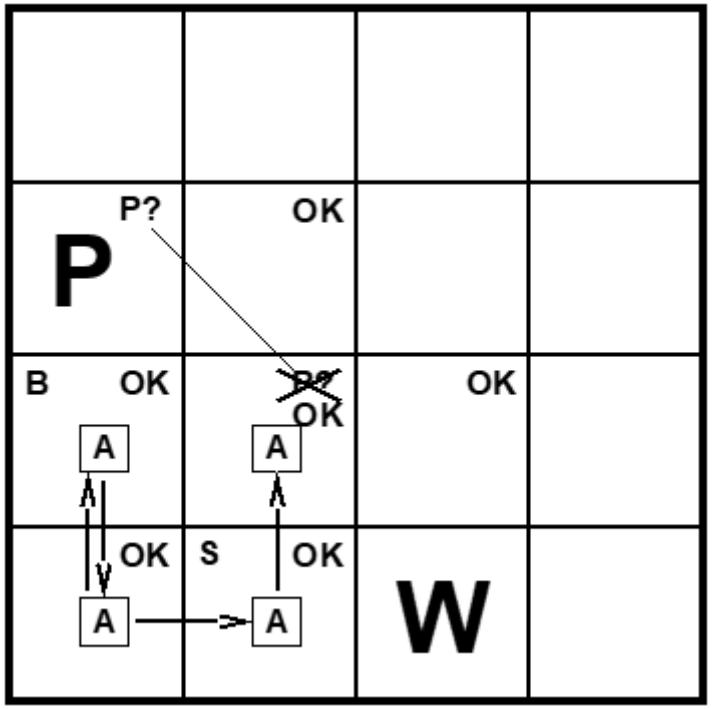
Exploring the Wumpus World



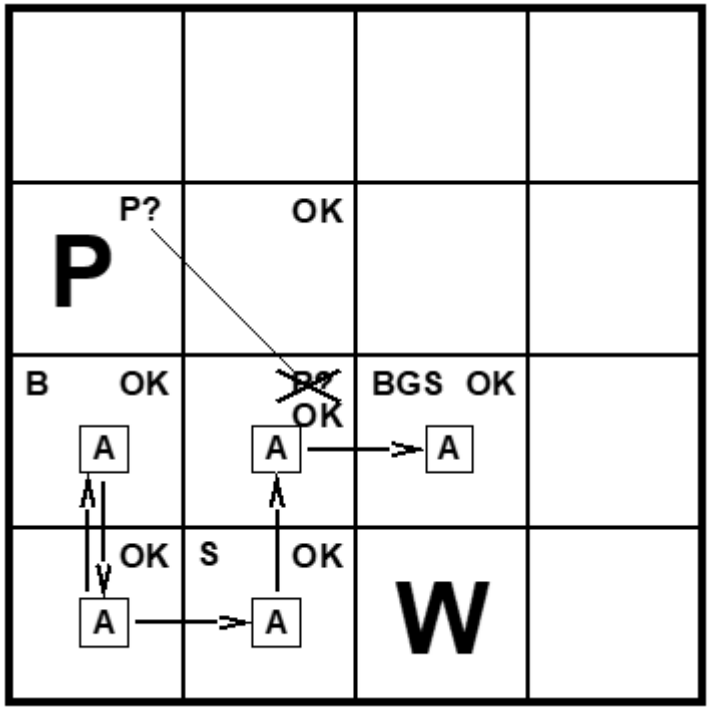
Exploring the Wumpus World



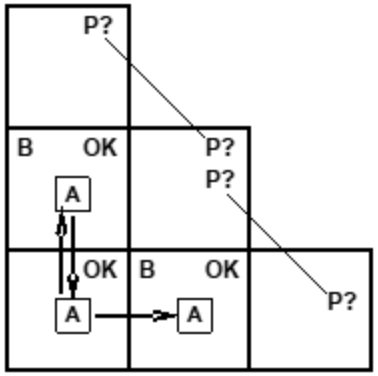
Exploring the Wumpus World



Exploring the Wumpus World

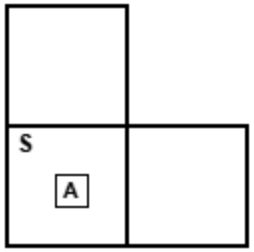


Other tight spots



Breeze in (1,2) and (2,1)
⇒ no safe actions

Assuming pits uniformly distributed,
(2,2) has pit w/ prob 0.86, vs. 0.31



Smell in (1,1)
⇒ cannot move
Can use a strategy of coercion:
shoot straight ahead
wumpus was there ⇒ dead ⇒ safe
wumpus wasn't there ⇒ safe

Logic in general

- Logics are formal languages for representing information such that conclusions can be drawn
- Syntax defines the sentences in the language
- Semantics defines the "meaning" of sentences, i.e., define truth of a sentence in a world
- E.g., the language of arithmetic
  - $x + 2 \geq y$ is a sentence; $x2 + y >$ is not a sentence
  - $x+ 2 \geq y$ is true iff the number $x+ 2$ is no less than the number y
  - $x+ 2 \geq y$ is true in a world where $x = 7, y = 1$
  - $x+ 2 \geq y$ is false in a world where $x = 0, y = 6$

Entailment

- Entailment means that one thing follows from another: $KB \vdash \alpha$
- Knowledge base KB entails sentence α

  If and only if

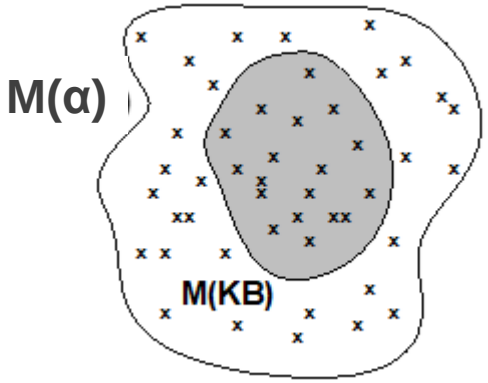
  α is true in all worlds where KB is true
- E.g., the KB containing "the Giants won" and "the Reds won" entails "Either the Giants won or the Reds won"
- E.g., $x + y = 4$ entails $4 = x + y$
- Entailment is a relationship between sentences (i.e., syntax) that is based on semantics.

Models

- Logicians typically think in terms of models, which are formally structured worlds with respect to which truth can be evaluated
- We say m is a model of a sentence α if α is true in m
- M(α) is the set of all models of α
- Then KB ⊨ α if and only if M(KB) ⊆ M(α)

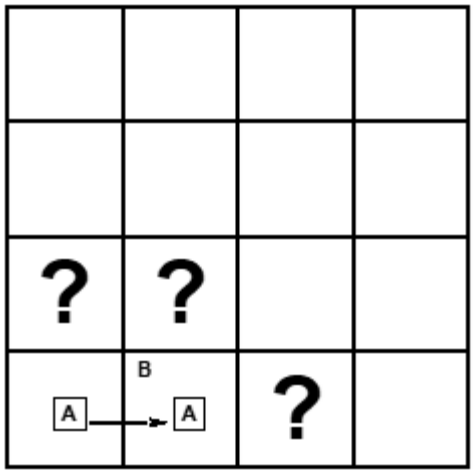
E.g. KB = Giants won and Reds won

α = Giants won

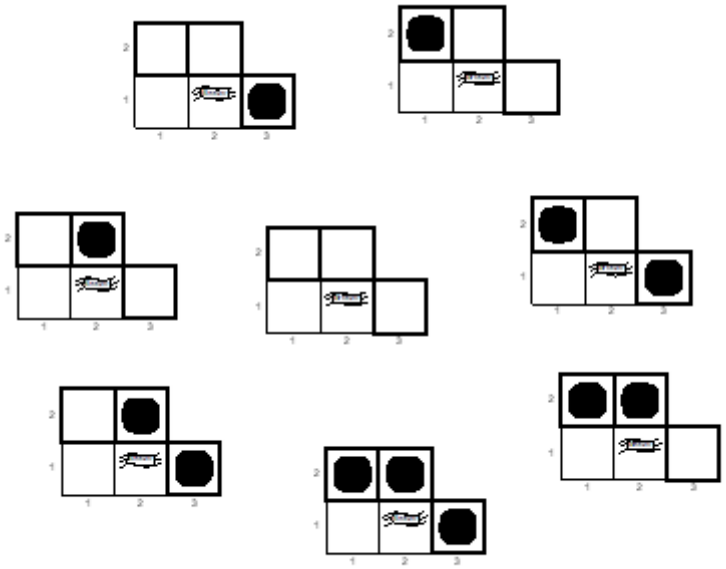


Entailment in the Wumpus World

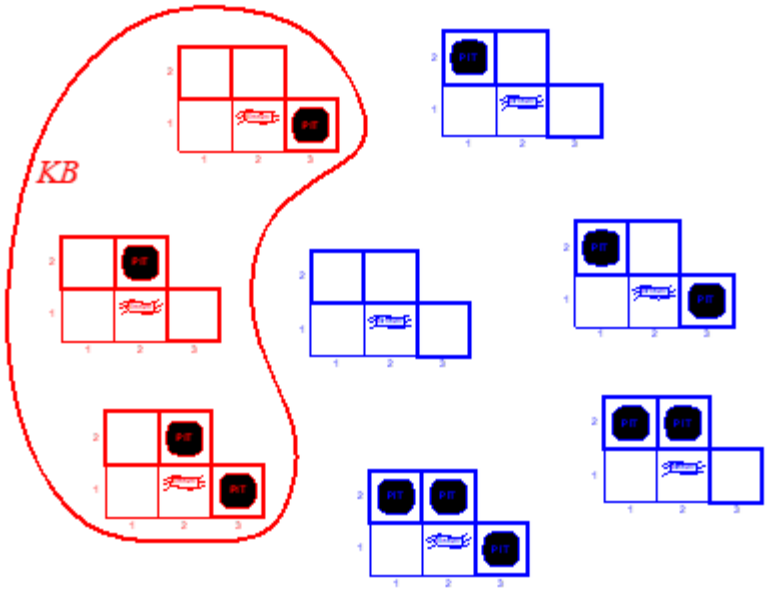
- Situation after detecting nothing in [1,1], moving right, breeze in [2,1]
- Consider possible models for ?s assuming only pits
- 3 Boolean choices => 8 possible models



Wumpus Models

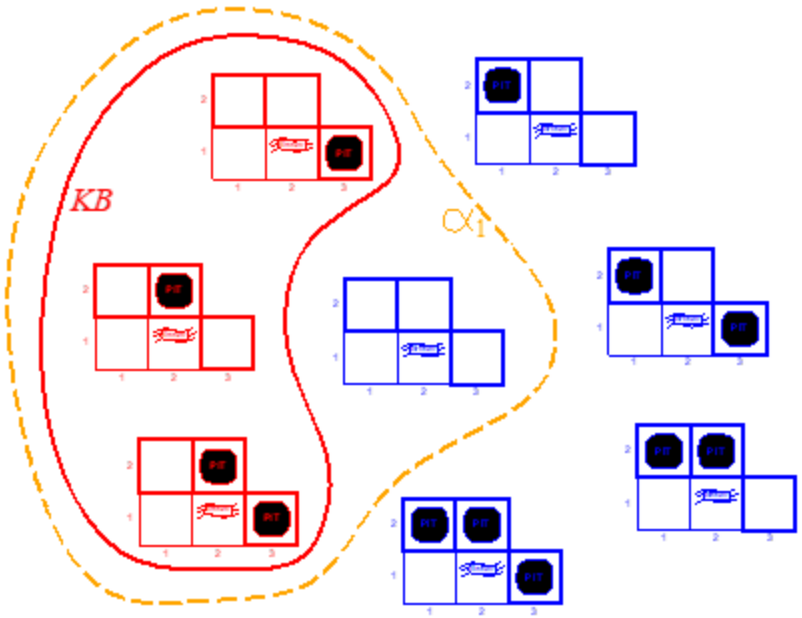


Wumpus Models



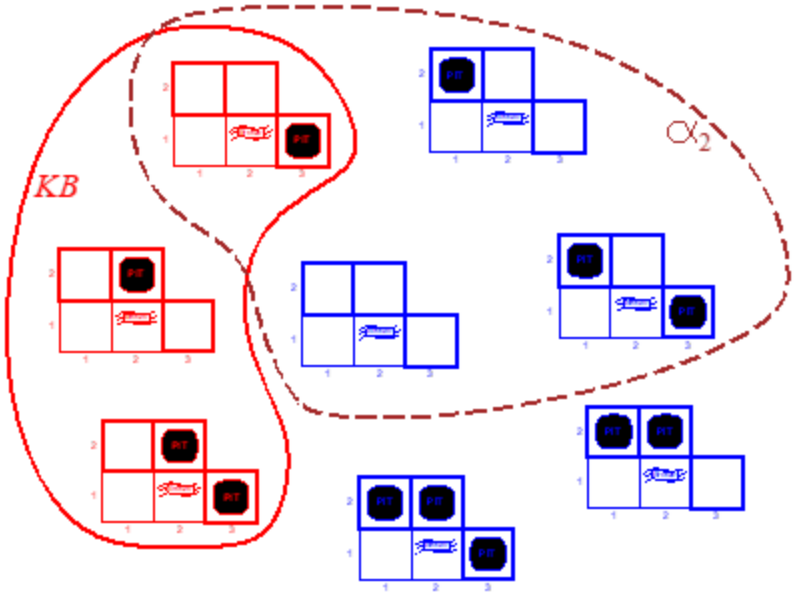
- *KB* = wumpus-world rules + observations

Wumpus Models



- KB = wumpus-world rules + observations
- α_1 = "[1,2] is safe", KB $\models \alpha_1$, proved by model checking

Wumpus Models



- KB = wumpus-world rules + observations
- α_2 = "[2,2] is safe", KB $\not\models \alpha_2$

Propositional Logic: Syntax

- Propositional logic is the simplest logic – illustrates basic ideas
- The proposition symbols P_1,P_2, etc are sentences
- If S is a sentence, ¬S is a sentence (negation)
- If S_1 and S_2 are sentences, $S_1 \wedge S_2$ is a sentence (conjunction)
- If S_1 and S_2 are sentences, $S_1 \vee S_2$ is a sentence (disjunction)
- If S_1 and S_2 are sentences, $S_1 \Rightarrow S_2$ is a sentence (implication)
- If S_1 and S_2 are sentences, $S_1 \Leftrightarrow S_2$ is a sentence (biconditional)

Propositional Logic: Semantics

- Each model specifies true/false for each proposition symbol
- E.g., $P_{1,2}$ $P_{2,2}$ $P_{3,1}$
  false false true

(With these symbols, 8 possible models can be enumerated automatically)

- Rules for evaluating truth with respect to a model m:

$\neg S$	is true iff	S	is false			
$S_1 \wedge S_2$	is true iff	S_1	is true and	S_2	is true	
$S_1 \vee S_2$	is true iff	S_1	is true or	S_2	is true	
$S_1 \Rightarrow S_2$	is true iff	S_1	is false or	S_2	is true	
i.e.,	is false iff	S_1	is true and	S_2	is false	
$S_1 \Leftrightarrow S_2$	is true iff	$S_1 \Rightarrow S_2$	is true and	$S_2 \Rightarrow S_1$	is true	

Simple recursive process evaluates an arbitrary sentence, e.g.,
$\neg P_{1,2} \wedge (P_{2,2} \vee P_{3,1})$ = true $\wedge$ (false $\vee$ true) = true $\wedge$ true = true

Truth Tables for Connectives

P	Q	¬P	$P \wedge Q$	$P \vee Q$	$P \Rightarrow Q$	$P \Leftrightarrow Q$
false	false	true	false	false	true	true
false	true	true	false	true	true	false
true	false	false	false	true	false	false
true	true	false	true	true	true	true

Wumpus World Sentences

- Let $P_{i,j}$ be true if there is a pit in $[i,j]$.
- Let $B_{i,j}$ be true if there is a breeze in $[i,j]$.

$\neg P_{1,1}$

$\neg B_{1,1}$

$B_{2,1}$

"Pits cause breezes in adjacent squares"

Wumpus World Sentences

- Let $P_{i,j}$ be true if there is a pit in [i,j].
- Let $B_{i,j}$ be true if there is a breeze in [i,j].

Rules

R_1 $\neg P_{1,1}$

R_2 $\neg B_{1,1}$

R_3 $B_{2,1}$

"Pits cause breezes in adjacent squares"

R_4 $B_{1,1} \Leftrightarrow (P_{1,2} \vee P_{2,1})$

R_5 $B_{2,1} \Leftrightarrow (P_{1,1} \vee P_{2,2} \vee P_{3,1})$

"A square is breezy if and only if there is an adjacent pit"

These rules together form our KB

Truth Tables for Inference

$B_{1,1}$	$B_{2,1}$	$P_{1,1}$	$P_{1,2}$	$P_{2,1}$	$P_{2,2}$	$P_{3,1}$	R_1	R_2	R_3	R_4	R_5	KB
false	false	false	false	false	false	false	true	true	true	true	false	false
false	false	false	false	false	false	true	true	true	false	true	false	false
⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮
false	true	false	false	false	false	false	true	true	false	true	true	false
false	true	false	false	false	false	true	true	true	true	true	true	true
false	true	false	false	false	true	false	true	true	true	true	true	true
false	true	false	false	false	true	true	true	true	true	true	true	true
false	true	false	false	true	false	false	true	false	false	true	true	false
⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮
true	true	true	true	true	true	true	false	true	true	false	true	false

- Enumerate rows (different assignments to symbols), if KB is true in row, check that α is too
- KB = rules derived from observations, laws of Wumpus
- α = what we're interested in testing/checking

Truth Tables for Inference

	$B_{1,1}$	$B_{2,1}$	$P_{1,1}$	$P_{1,2}$	$P_{2,1}$	$P_{2,2}$	$P_{3,1}$	R_1	R_2	R_3	R_4	R_5	KB
	false	false	false	false	false	false	false	true	true	true	true	false	false
	false	false	false	false	false	false	true	true	true	false	true	false	false
	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮
	false	true	false	false	false	false	false	true	true	false	true	true	false
KB	false	true	false	false	false	false	true	true	true	true	true	true	true
KB	false	true	false	false	false	true	false	true	true	true	true	true	true
KB	false	true	false	false	false	true	true	true	true	true	true	true	true
	false	true	false	false	true	false	false	true	false	false	true	true	false
	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮
	true	true	true	true	true	true	true	false	true	true	false	true	false

- Enumerate rows (different assignments to symbols), if KB is true in row, check that α is too
- KB = rules derived from observations, laws of Wumpus
- α = what we're interested in testing/checking

Pros and Cons of Propositional Logic

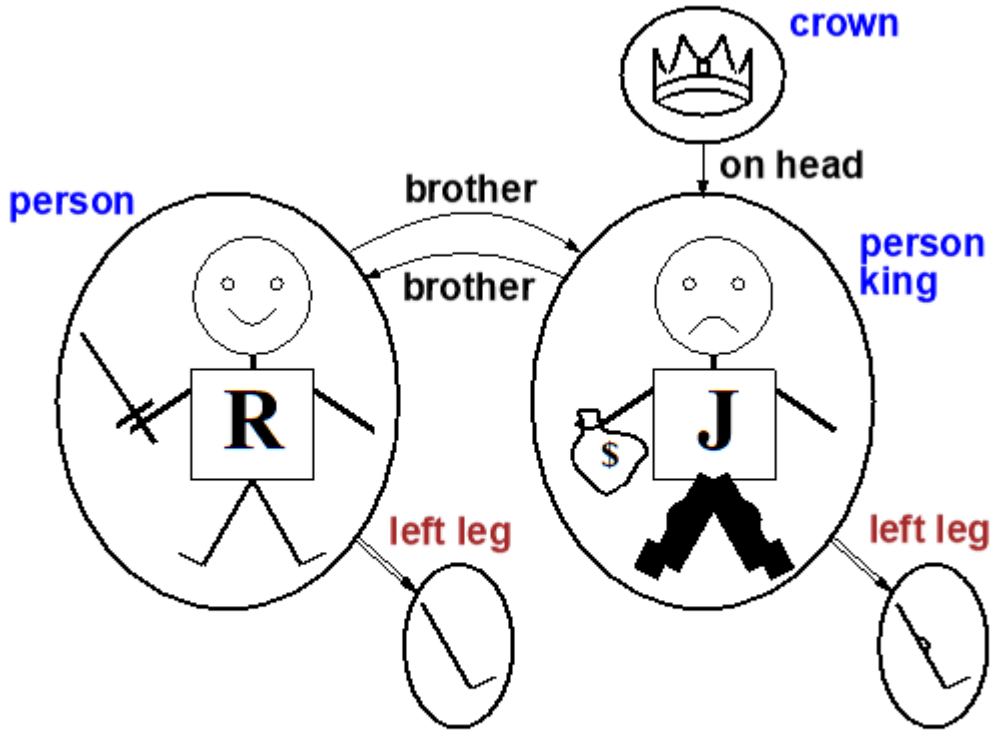
- ☺ Propositional logic is declarative: pieces of syntax correspond to facts
- ☺ Propositional logic allows partial/disjunctive/negated information (unlike most data structures and databases)
- ☺ Propositional logic is compositional: meaning $B_{1,1} \wedge P_{1,2}$ is derived from meaning of $B_{1,1}$ and of $P_{1,2}$
- ☺ Meaning in propositional logic is context-independent (unlike natural language, where meaning depends on context)
- ☹ Propositional logic has very limited expressive power (unlike natural language)

E.g., cannot say "pits cause breezes in adjacent squares" except by writing one sentence for each square

First-order Logic

- Whereas propositional logic assumes world contains facts, first-order logic (like natural language) assumes the world contains
  - Objects: people, houses, numbers, theories, Ronald McDonald, colors, baseball games, wars, centuries…
  - Relations: red, round, bogus, prime, multistoried… , brother of, bigger than, inside, part of, has color, occurred after, owns, comes between…
  - Functions: father of, best friend, third inning of, one more than, end of…

Models for First-order Logic: Example



Syntax of FOL: Basic Elements

- Constants — KingJohn, 2, UCB, …
- Predicates — Brother, >, …
- Functions — Sqrt, LeftLegOf, …
- Variables — *x, y, a, b, …*
- Connectives — ∧ ∨ ¬ ⇒ ⇔
- Equality — =
- Quantifiers — ∀ ∃

Atomic Sentences

- Atomic sentence = *predicate*($term_1$, …, $term_m$)

  or $term_1 = term_2$

- Term = *function*($term_1$, …, $term_m$)

  or *constant* or *variable*

E.g., *Brother(KingJohn, RichartTheLionheart)*

> (Length(LeftLegOf(Richard)), Length(LeftLegOf(KingJohn)))

Complex Sentences

- Complex sentences are made from atomic sentences using connectives

$\neg S$, $S_1 \wedge S_2$, $S_1 \vee S_2$, $S_1 \Rightarrow S_2$, $S_1 \Leftrightarrow S_2$

E.g.

Sibling(KingJohn, Richard) ⇒ Sibling(Richard, KingJohn)

>(1,2) V ≤ (1,2)

>(1,2) Λ ¬> (1,2)

Models for FOL: Lots!

- Entailment in propositional logic can be computed by enumerating models
- We can enumerate the FOL models for a given KB vocabulary:

For each number of domain elements n from 1 to ∞

For each k-ary predicate P_k in the vocabulary

For each possible k-ary relation on n objects

For each constant symbol C in the vocabulary

For each choice of referent for C from n objects…

Computing entailment by enumerating FOL models is not easy!

Universal Quantification

- ∀ *<variables> <sentence>*
- Everyone at Berkeley is smart:
  - ∀*x* *At(x, Berkeley) ⇒ Smart(x)*
- ∀*x* *P* is true in a model *m* iff *P* is true with *x* being each possible object in the model
- Roughly speaking, equivalent to the conjunction of instantiations of P

(At(KingJohn, Berkeley) ⇒ Smart(KingJohn))

∧ (At(Richard, Berkeley) ⇒ Smart(Richard))

∧ (At(Berkeley, Berkeley) ⇒ Smart(Berkeley)) …

A common mistake to avoid

- Typically, $\Rightarrow$ is the main connective with $\forall$
- Common mistake, using $\wedge$ as the main connective with $\forall$:

$$\forall x \;\; At(x, Berkeley) \wedge Smart(x)$$

Means "Everyone is at Berkeley and everyone is smart"

Existential Quantification

- ∃ *<variable> <sentence>*
- Someone at Stanford is smart:
  - *∃x At(x, Stanford) ∧ Smart(x)*
- *∃x P* is true in a model *m* iff *P* is true with *x* being some possible object in the model
- Roughly speaking, equivalent to the disjunction of instantiations of *P*

(At(KingJohn, Stanford) ∧ Smart(KingJohn))

∨ (At(Richard, Stanford) ∧ Smart(Richard))

∨ …

Another common mistake to avoid

- Typically, $\wedge$ is the main connective with $\exists$
- Common mistake: using $\Rightarrow$ as the main connective with $\exists$:

$$\exists x \; At(x, Stanford) \Rightarrow Smart(x)$$

Is true if there is anyone who is not at Stanford!

Properties of quantifiers

- $\forall x\ \forall y$ is the same as $\forall y\ \forall x$
- $\exists x\ \exists y$ is the same as $\exists y\ \exists x$
- $\exists x\ \forall y$ is not the same as $\forall y\ \exists x$
- $\exists x\ \forall y\ \ Loves(x,y)$
  - "There is a person who loves everyone in the world"
- $\forall y\ \exists x\ \ Loves(x,y)$
  - "Everyone in the world is loved by at least one person"
- Quantifier duality: each can be expressed using the other
  - $\forall x\ \ Likes(x, IceCream)\ \ \neg\exists x\ \neg Likes(x, IceCream)$
  - $\exists x\ Likes(x, Broccoli)\ \ \ \ \neg\forall x\ \neg Likes(x, Broccoli)$

Fun with Sentences

- Brothers are siblings

Fun with Sentences

Brothers are siblings

- $\forall x,y\ Brother(x,y) \Rightarrow Sibling(x,y).$

"Sibling" is symmetric

Fun with Sentences

Brothers are siblings

- $\forall x,y\ Brother(x,y) \Rightarrow Sibling(x,y).$

"Sibling" is symmetric

- $\forall x,y\ Sibling(x,y) \Leftrightarrow Sibling(y,x).$

One's mother is one's female parent

Fun with Sentences

Brothers are siblings

- $\forall x,y\ Brother(x,y) \Rightarrow Sibling(x,y).$

"Sibling" is symmetric

- $\forall x,y\ Sibling(x,y) \Leftrightarrow Sibling(y,x).$

One's mother is one's female parent

- $\forall x,y\ Mother(x,y) \Leftrightarrow (Female(x) \wedge (Parent(x,y)).$

A first cousin is a child of a parent's sibling

Fun with Sentences

Brothers are siblings

- $\forall x,y\ Brother(x,y) \Rightarrow Sibling(x,y).$

"Sibling" is symmetric

- $\forall x,y\ Sibling(x,y) \Leftrightarrow Sibling(y,x).$

One's mother is one's female parent

- $\forall x,y\ Mother(x,y) \Leftrightarrow (Female(x) \wedge (Parent(x,y)).$

A first cousin is a child of a parent's sibling

- $\forall x,y\ FirstCousin(x,y) \Leftrightarrow \exists px,py\ \ Parent(px,x) \wedge Sibling(px,py) \wedge Parent(py,y)$

Interacting with FOL KBs

Suppose a wumpus-world is using a FOL KB and perceives a smell and a breeze (but no glitter) at *t = 5.*

Tell(KB, Percept([Smell, Breeze, None],5))

Ask(KB, ∃a Action(a,5))

I.e., does *KB* entail any particular actions at *t = 5*?

Answer: *Yes, {a/Shoot}* ← substitution (binding list)

Given a sentence *S* and a substitution *σ*,

Sσ denotes that result of plugging *σ* into *S*; e.g.,

S = Smarter(x,y)

σ = {x/Hillary, y/Bill}

Sσ = Smarter(Hillary, Bill)

Ask (KB, S) returns some/all σ such that KB⊨Sσ

Deducing Hidden Properties

Properties at locations:

$\forall x,t \;\; At(Agent,x,t) \wedge Smelt(t) \Rightarrow Smelly(x)$

$\forall x,t \;\; At(Agent,x,t) \wedge Breeze(t) \Rightarrow Breezy(x)$

Squares are breezy near a pit:

Diagnostic rule – infer cause from effect

$\forall y \; Breezy(y) \Rightarrow \exists x \; Pit(x) \wedge Adjacent(x,y)$

Causal rule – infer effect from cause

$\forall x,y \; Pit(x) \wedge Adjacent(x,y) \Rightarrow Breezy(y)$

Neither of these is complete – e.g., the causal rule doesn't say whether squares far away from pits can be breezy

Definition for the Breezy predicate:

$$\forall y \; Breezy(y) \;\; \Leftrightarrow [\exists x \; Pit(x) \wedge Adjacent(x,y)]$$

Describing Actions

- "Effect" axiom – describes changes due to action

$\forall s\ AtGold(s) \Rightarrow Holding(Gold, Result(Grab,s))$

- "Frame" axiom – describes non-changes due to action

$\forall s\ HaveArrow(s) \Rightarrow HaveArrow(Result(Grab,s))$

- Frame problem: find an elegant way to handle non-change
  - (a) representation – avoid frame axioms
  - (b) inference – avoid repeated "copy-overs" to keep track of state
- Qualification problem: true descriptions of real actions required endless caveats – what if gold is slippery or nailed down or …
- Ramification problem: real actions have many secondary consequences – what about the dust on the gold, wear and tear on gloves, …

Making Plans

- Initial condition in KB:

  $At(Agent, [1,1], S_0)$

  $At(Gold, [1,2], S_0)$

- Query: $Ask(KB, \exists s\ Holding(Gold,s))$
  - i.e., in what situation will I be holding the gold?
- Answer: $\{s/Result(Grab, Result(Forward, S_0))\}$
  - i.e., go forward and then grab the gold
- This assumes that the agent is interested in plans starting at S_0 and that S_0 is the only situation described in KB

Making Plans: A better way

- Represent plans as action sequences $[a_1, a_2, \ldots, a_n]$

PlanResult(p,s) is the result of executing *p* in *s*

- Then the query $Ask(KB, \exists p \;\; Holding(Gold, PlanResult(p,S_0)))$ has the solution $\{p/[Forward,Grab]\}$
- Definition of *PlanResult* in terms of *Result*:

$\forall s \;\; PlanResult([],s) = s$

$\forall a,p,s \;\; PlanResult([a|p],s) = PlanResult(p,Result(a,s))$

- Planning systems are special-purpose reasoners designed to do this type of inference more efficiently than a general-purpose reasoner

Summary

- Predicate logic
  - Basic representation for factual knowledge
- First-order logic
  - Objects and relations are semantic primitives
  - Syntax: constants, functions, predicates, equality, quantifiers
  - Situation calculus:
    - Conventions for describing actions and changes in FOL
    - Can formulate planning as inference on a situation calculus KB
  - Increased expressive power over propositional logic
    - … or at least, sufficient to define Wumpus world